Security in distributed systems

Distributed Systems

Alberto Lafuente, Mikel Larrea

UPV/EHU

Contents

- Introduction
- Threats and attacks
- Security policies
- Security mechanisms
- Authentication protocols
  - Examples: Needham-Schroeder, Kerberos
- Secure Multiparty Computation

Bibliography

- [COU05] Chapter 7
- [TAN06] Chapter 9
- Yao, A. *Protocols for secure computations*. In Proceedings of the twenty-third annual IEEE Symposium on Foundations of Computer Science, pages 160-164. IEEE Computer Society, 1982
- Lysyanskaya, A. *Protección de secretos*. Investigación y Ciencia, Nov 2008, pages 75-81

Introduction

- Security policies:
  - they define specific limits that are set when sharing resources
    - example: who can enter into a given room
  - independent of technology
- Security mechanisms:
  - they define how security policies are implemented
    - example: lock + key, or a guard at the door
  - set of techniques dependent of technology

Threats and attacks

- Message interception:
  - listening to messages
- Message interruption:
  - message delay, denial of service
- Message modification:
  - alteration or corruption of messages
- Message fabrication:
  - identity impersonation (client or server)

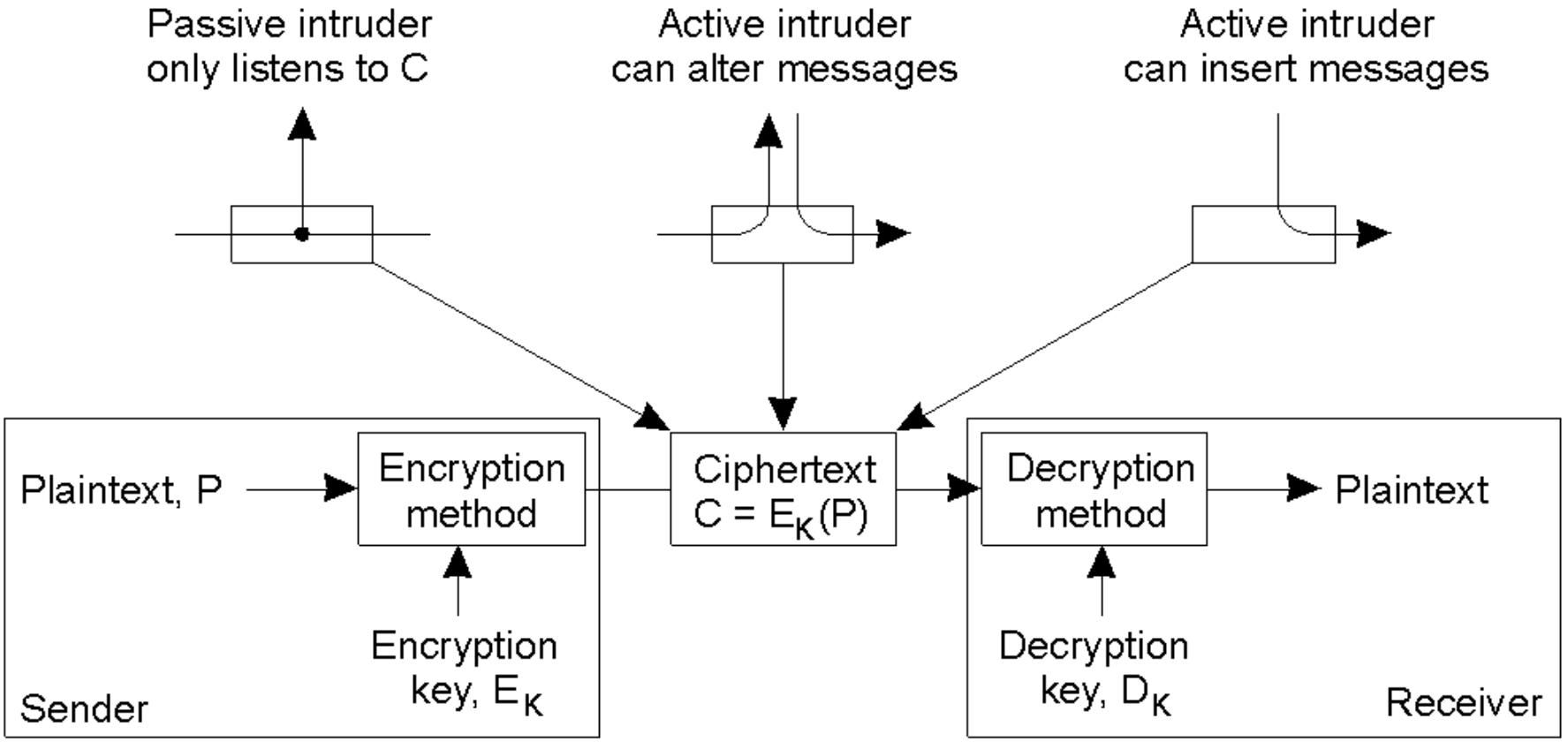
Security policies

- Confidentiality:
  - the information will be available only to authorized users
- Integrity:
  - the modifications of the information will only be made by the authorized users
- Access verification:
  - limit as much as possible permissions and access rights
  - limit the life (duration) and scope of access keys

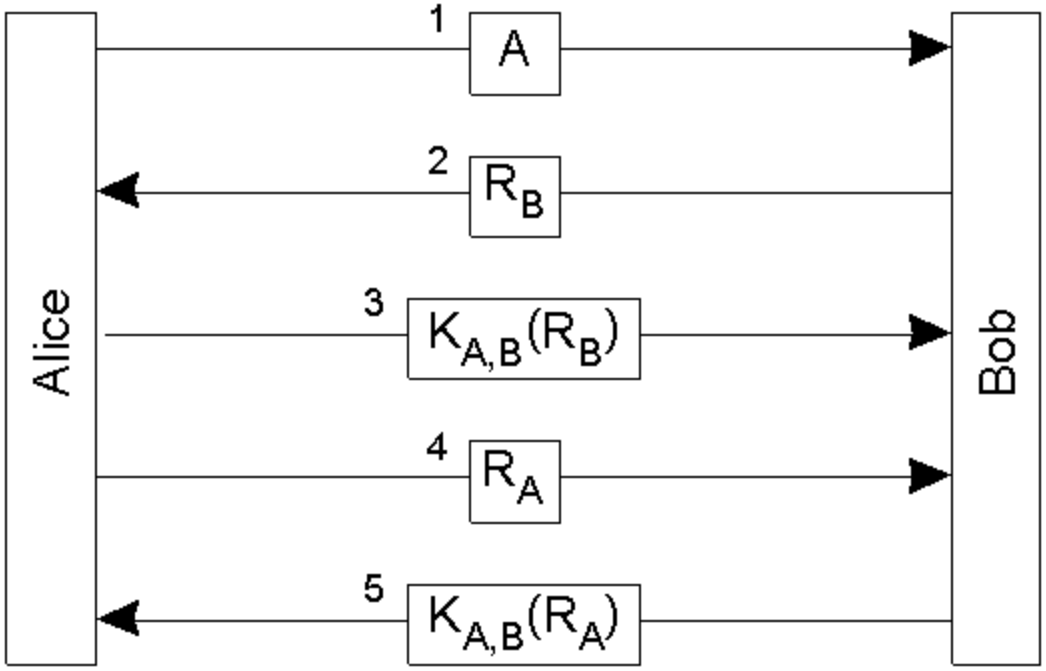
Security mechanisms

- Encryption
  - symmetric (secret key)
  - asymmetric (public and private key pair)
- Authentication
  - passwords
  - challenge-response protocols
- Authorization
  - access control lists, credentials
  - firewalls
- Audit
  - maintenance and analysis of traces (*logs*)

Encryption

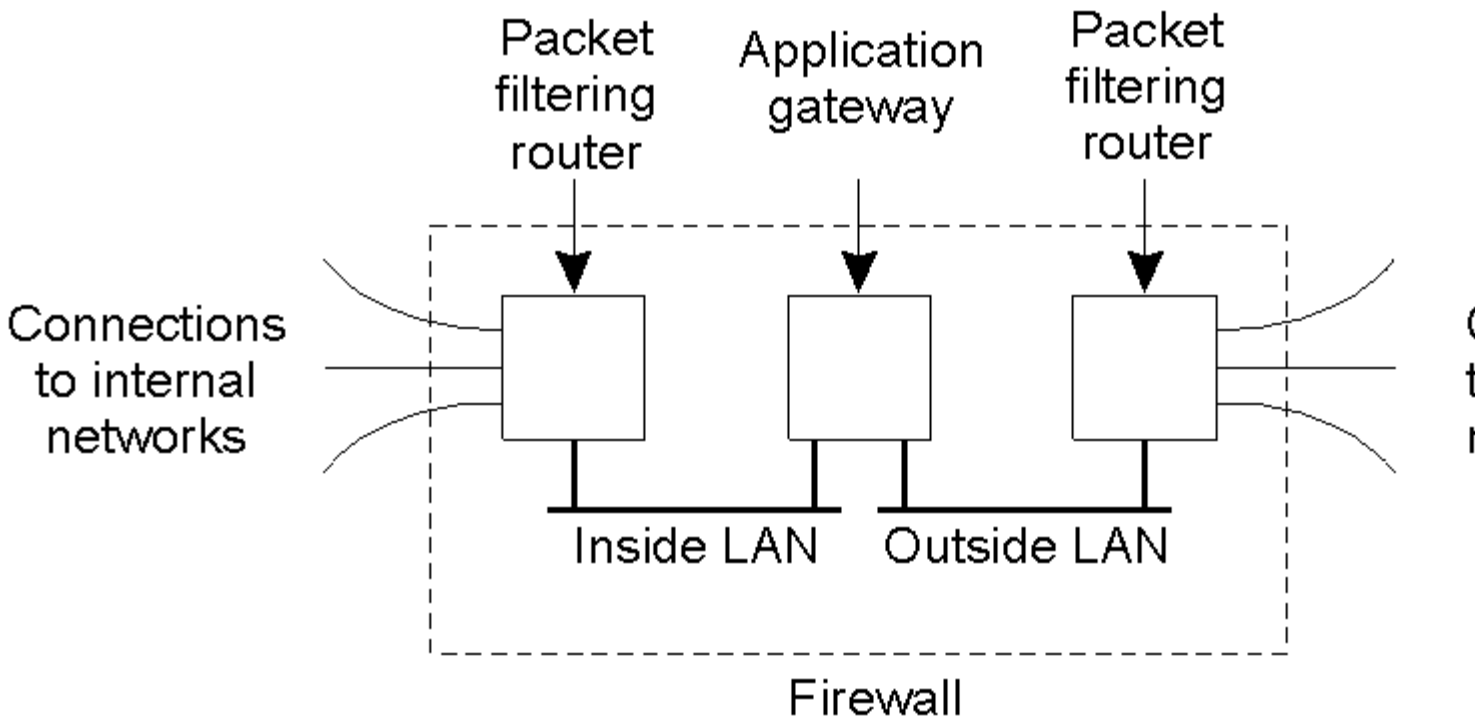

Passive intruder
only listens to C
Active intruder
can alter messages
Active intruder
can insert messages
Plaintext, P
Encryption
method
Encryption
key, E_K
Sender
Ciphertext
$C = E_K(P)$
Decryption
method
Decryption
key, D_K
Plaintext
Receiver

Authentication



Authentication based on a shared secret key

Authorization



A common implementation of a firewall

Needham-Schroeder protocol

Header	*Message*	*Notes*
1. A->S:	A, B, N_A	A requests S to supply a key for communication with B.
2. S->A:	$\{N_A, B, K_{AB}, \{K_{AB}, A\}_{K_B}\}_{K_A}$	S returns a message encrypted in A's secret key, containing a newly generated key K_{AB} and a 'ticket' encrypted in B's secret key. The nonce N_A demonstrates that the message was sent in response to the preceding one. A believes that S sent the message because only S knows A's secret key.
3. A->B:	$\{K_{AB}, A\}_{K_B}$	A sends the 'ticket' to B.
4. B->A:	$\{N_B\}_{K_{AB}}$	B decrypts the ticket and uses the new key K_{AB} to encrypt another nonce N_B.
5. A->B:	$\{N_B - 1\}_{K_{AB}}$	A demonstrates to B that it was the sender of the previous message by returning an agreed transformation of N_B.

Kerberos

- Authentication protocol developed at MIT in the 1980s
- Source code available from MIT (www.mit.edu)
- Included in the OSF DCE, NFS, AFS-3, Microsoft Windows

Kerberos - Architecture

- System architecture:
  - **Kerberos Key Distribution Centre** (**KDC**). It is composed by the following two services:
    - **Authentication Service** (**AS**): authenticates clients on login, and supplies tickets to access the Ticket-Granting Service
    - **Ticket-Granting Service** (**TGS**): supplies tickets and session keys to clients for accessing particular services
- Kerberos deals with three kinds of security objects: tickets, authenticators, and session keys

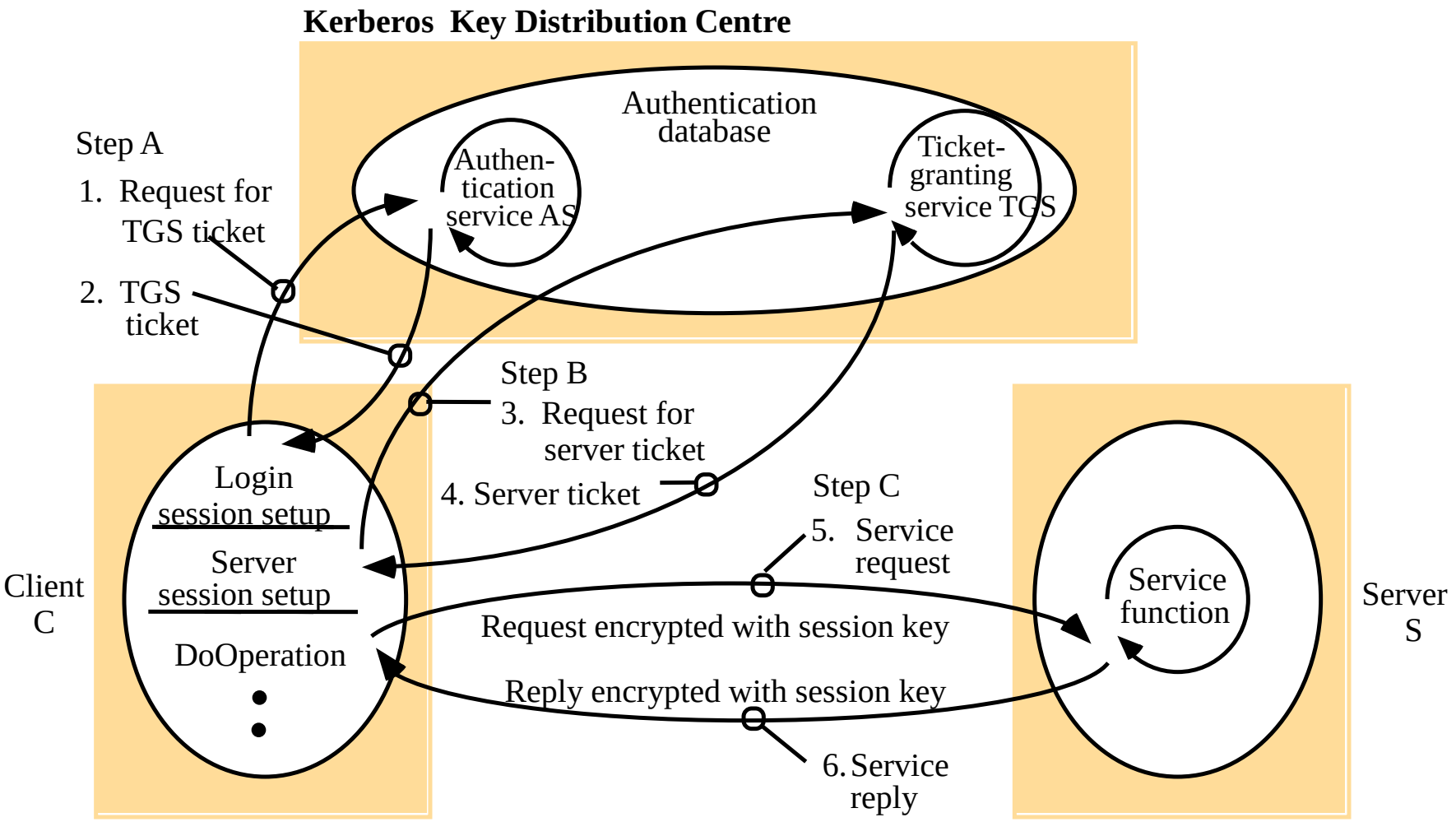
Kerberos - Architecture

- **Ticket**: a token issued to a client by the Kerberos Ticket-Granting Service for presentation to a particular server, verifying that the client has recently been authenticated by Kerberos. Tickets include an expiry time and a session key

- **Authenticator**: a token constructed by a client and sent to a server to prove the identity of the client and the recency of the communication (single use)

- **Session key**: a secret key randomly generated by the Kerberos Authentication Service and issued to a client for use when communicating with a particular server

Kerberos - Protocol



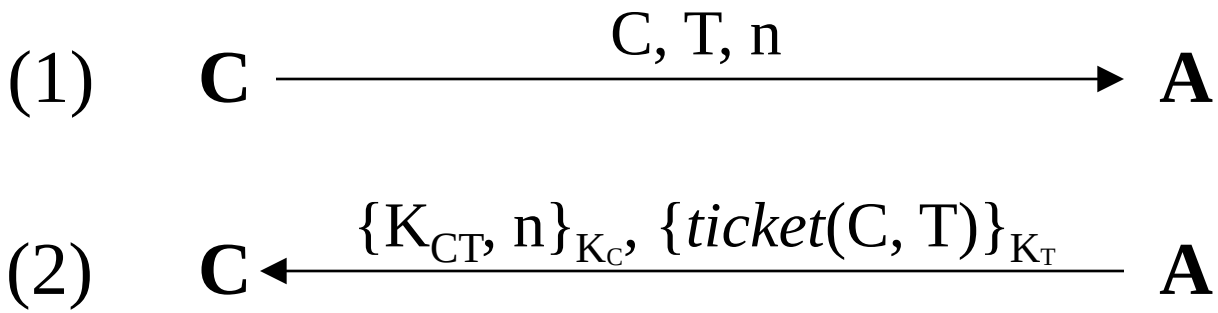
Kerberos - Protocol

- Notation
  - $\{M\}_k$: message M encrypted with key K
  - K_C: secret key of client C
  - *ticket*(C, S) = (C, S, t_1, t_2, K_{CS})
    - t_1: begin of validity period for the ticket
    - t_2: end of validity period for the ticket
    - K_{CS}: session key between C and S (randomly generated)
  - *authent*(C) = (C, t)
    - t: timestamp
  - n: number to identify every message
  - A: name of AS
  - T: name of TGS
  - K_T: secret key of TGS

Kerberos - Protocol

Step A: Login session setup

Getting a session key and a ticket for TGS

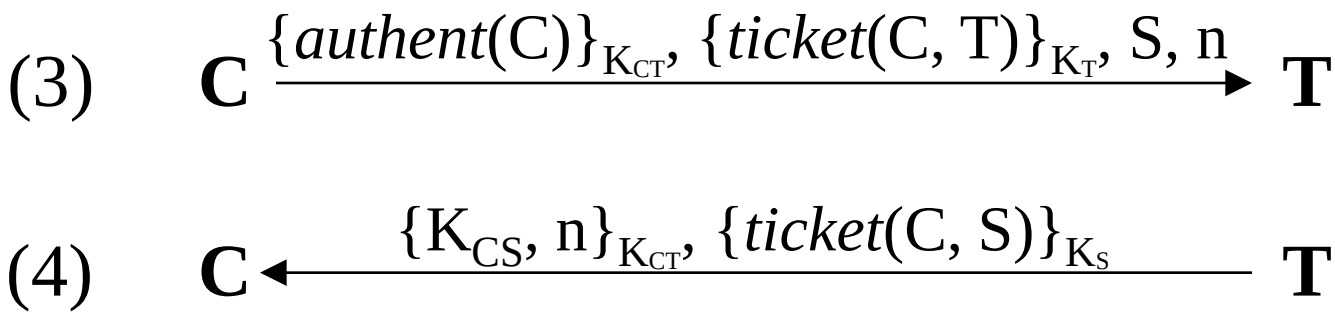


Client C is able to decrypt the part of message (2) which is encrypted with its secret key K_C

Kerberos - Protocol

Step B: Server session setup

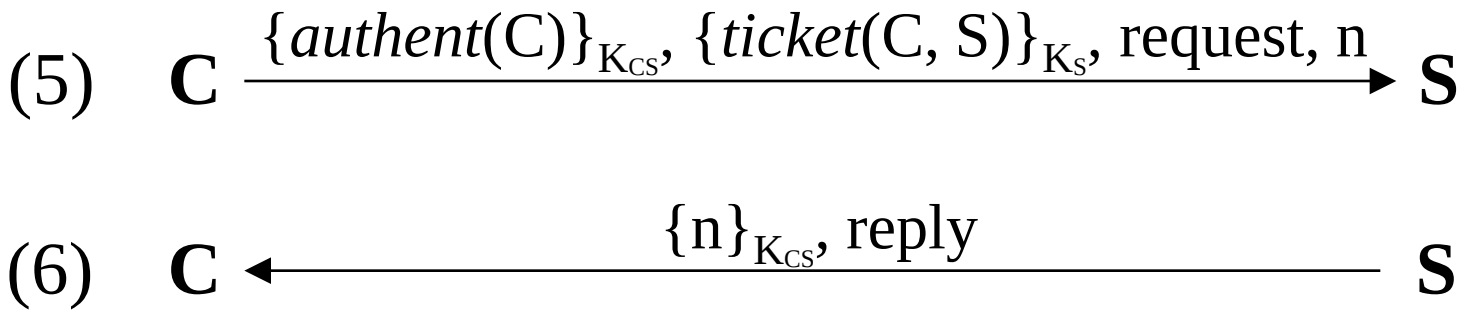
Getting a session key and a ticket for S



Kerberos - Protocol

Step C: Accessing the service

Accessing the server S: request/reply



If needed, request and reply could be encrypted with K_{CS}. Including the value n in message (6) allows C to ensure the authenticity of S

Secure Multiparty Computation

- The problem:
  - Let's calculate how much we weight…
  - …without knowing each other's weight ☺
- Approaches:
  - Centralized: by means of a *Trusted Third Party* (TTP) entity
  - Distributed: by means of a *Secure Multiparty Computation* (SMC) protocol

Secure Multiparty Computation

Model

- n participants: $P_1, \ldots, P_n$
- Each participant P_i has a private input x_i
- Goal:
  - Compute a function $f(x_1, \ldots, x_n) = (y_1, \ldots, y_n)$, such that P_i obtains y_i but no other information, in particular no x_i
- Even if some participants behave maliciously:
  - Initially, Byzantine failure model
  - Using hardware security modules, e.g., smartcards, omission failure model (easier to handle)

Secure Multiparty Computation

Example

- Three participants, P_1, P_2, P_3. Each one has a private value A_i
- Function to compute:
  - $f(A_1, A_2, A_3) = A_1+A_2+A_3$
- Strategy:
  - Each participant chooses 3 values between 0 and 1000: two of them are randomly chosen, and the third one is such that the sum of the three values modulo 1000 is equal to A_i
    - For example, if A_i=54 ➔ 300, 550, 204

Secure Multiparty Computation

Example

P_1	P_2	P_3
300	700	320
550	180	500
204	197	239
1054	**1077**	**1059**

Secure Multiparty Computation

Example

P_1	P_2	P_3
300	700	320
180	500	550
239	204	197

Every participant distributes two of its three values among the other participants, and keeps the third value

Secure Multiparty Computation

Example

P_1	P_2	P_3
300	700	320
180	500	550
239	204	197
719	**404**	**67**

Every participant sums the two values received plus the value kept, modulo 1000

Secure Multiparty Computation

Example

P_1	P_2	P_3
719	719	719
404	404	404
67	67	67
190	**190**	**190**

Finally, every participant broadcasts the result obtained previously, and repeats the addition for the received three values (modulo 1000)